

# Rappel JavaScript ES6

## Prérequis pour React

M204 - Développement Front-End

### Pourquoi ce Rappel ?

React utilise intensivement les fonctionnalités **JavaScript ES6+** (ECMAScript 2015 et versions ultérieures).

**Sans maîtriser ES6, il sera difficile de :**

- Comprendre le code React
- Écrire des composants
- Manipuler les props et le state
- Utiliser les méthodes de tableaux (map, filter, etc.)

**Ce document** révisé les concepts JavaScript ES6 essentiels pour React.

## Table des matières

|          |                                                   |          |
|----------|---------------------------------------------------|----------|
| <b>1</b> | <b>let et const (Variables Modernes)</b>          | <b>3</b> |
| 1.1      | Ancien : var (À Éviter) . . . . .                 | 3        |
| 1.2      | Moderne : let et const . . . . .                  | 3        |
| 1.3      | Attention avec les Objets et Tableaux . . . . .   | 3        |
| 1.4      | Quelle Règle Suivre ? . . . . .                   | 4        |
| <b>2</b> | <b>Arrow Functions (Fonctions Flèches)</b>        | <b>5</b> |
| 2.1      | Ancienne Syntaxe . . . . .                        | 5        |
| 2.2      | Nouvelle Syntaxe : Arrow Functions . . . . .      | 5        |
| 2.3      | Exemples Pratiques . . . . .                      | 5        |
| 2.4      | Utilisation dans React . . . . .                  | 6        |
| 2.5      | Différence avec function . . . . .                | 6        |
| <b>3</b> | <b>Template Literals (Chaînes Modernes)</b>       | <b>7</b> |
| 3.1      | Ancienne Méthode : Concaténation . . . . .        | 7        |
| 3.2      | Nouvelle Méthode : Template Literals . . . . .    | 7        |
| 3.3      | Chaînes Multi-Lignes . . . . .                    | 7        |
| 3.4      | Utilisation dans React . . . . .                  | 7        |
| <b>4</b> | <b>Destructuring (Déstructuration)</b>            | <b>9</b> |
| 4.1      | Destructuring d'Objets . . . . .                  | 9        |
| 4.2      | Renommer lors du Destructuring . . . . .          | 9        |
| 4.3      | Valeurs par Défaut . . . . .                      | 9        |
| 4.4      | Destructuring de Tableaux . . . . .               | 9        |
| 4.5      | Utilisation dans React (TRÈS IMPORTANT) . . . . . | 10       |

|          |                                              |           |
|----------|----------------------------------------------|-----------|
| <b>5</b> | <b>Spread Operator (...)</b>                 | <b>11</b> |
| 5.1      | Avec les Tableaux . . . . .                  | 11        |
| 5.2      | Avec les Objets . . . . .                    | 11        |
| 5.3      | Utilisation dans React (ESSENTIEL) . . . . . | 11        |
| <b>6</b> | <b>Méthodes de Tableaux (Array Methods)</b>  | <b>13</b> |
| 6.1      | map() - Transformer un Tableau . . . . .     | 13        |
| 6.2      | filter() - Filtrer un Tableau . . . . .      | 13        |
| 6.3      | find() - Trouver un Élément . . . . .        | 14        |
| 6.4      | reduce() - Réduire à une Valeur . . . . .    | 14        |
| 6.5      | Chaînage de Méthodes . . . . .               | 14        |
| <b>7</b> | <b>Modules (Import/Export)</b>               | <b>15</b> |
| 7.1      | Export . . . . .                             | 15        |
| 7.2      | Import . . . . .                             | 15        |
| 7.3      | Utilisation dans React . . . . .             | 15        |
| <b>8</b> | <b>Ternaire et Short-Circuit</b>             | <b>17</b> |
| 8.1      | Opérateur Ternaire . . . . .                 | 17        |
| 8.2      | Short-Circuit () . . . . .                   | 17        |
| <b>9</b> | <b>Récapitulatif : ES6 pour React</b>        | <b>18</b> |

# 1 let et const (Variables Modernes)

## 1.1 Ancien : var (À Éviter)

Avant ES6, on utilisait `var` :

```
1 var nom = "Ahmed";  
2 var age = 20;
```

**Problèmes avec `var` :**

- Portée (scope) de fonction, pas de bloc
- Peut être redéclaré par erreur
- "Hoisting" confus

## 1.2 Moderne : let et const

Règle Moderne

- **`const`** : Variable constante (ne peut pas être réassignée)
- **`let`** : Variable qui peut changer
- **`var`** : NE PLUS UTILISER !

```
1 // const : valeur qui ne change pas  
2 const nom = "Ahmed";  
3 const PI = 3.14159;  
4  
5 // let : valeur qui peut changer  
6 let age = 20;  
7 age = 21; // OK  
8  
9 // Erreur avec const  
10 const ville = "Agadir";  
11 ville = "Casablanca"; // ERREUR !
```

## 1.3 Attention avec les Objets et Tableaux

`const` empêche la **réassignation**, pas la **modification** :

```
1 // OK : Modifier le contenu d'un objet const  
2 const personne = { nom: "Ahmed", age: 20 };  
3 personne.age = 21; // OK  
4 personne.ville = "Agadir"; // OK  
5  
6 // ERREUR : Réassigner l'objet entier  
7 personne = { nom: "Sara" }; // ERREUR !  
8  
9 // Même chose avec les tableaux  
10 const fruits = ["Pomme", "Banane"];  
11 fruits.push("Orange"); // OK  
12 fruits = ["Mangue"]; // ERREUR !
```

## 1.4 Quelle Règle Suivre ?

### Best Practice

1. Par défaut, utilisez **const**
2. Utilisez **let** uniquement si la variable doit changer (compteurs, boucles)
3. N'utilisez **jamais var**

## 2 Arrow Functions (Fonctions Flèches)

### 2.1 Ancienne Syntaxe

```
1 // Fonction traditionnelle
2 function additionner(a, b) {
3     return a + b;
4 }
5
6 // Fonction anonyme
7 const additionner = function(a, b) {
8     return a + b;
9 };
```

### 2.2 Nouvelle Syntaxe : Arrow Functions

```
1 // Arrow function
2 const additionner = (a, b) => {
3     return a + b;
4 };
5
6 // Version courte (return implicite)
7 const additionner = (a, b) => a + b;
8
9 // Un seul paramètre : pas besoin de parenthèses
10 const carre = x => x * x;
11
12 // Aucun paramètre : parenthèses vides
13 const direBonjour = () => console.log("Bonjour !");
```

### 2.3 Exemples Pratiques

```
1 // Fonction classique
2 function multiplier(x) {
3     return x * 2;
4 }
5
6 // Équivalent en arrow function
7 const multiplier = x => x * 2;
8
9 // Avec plusieurs lignes
10 const calculer = (a, b) => {
11     const somme = a + b;
12     const produit = a * b;
13     return { somme, produit };
14 };
15
16 // Return implicite d'un objet (attention aux parenthèses)
17 const creerPersonne = (nom, age) => ({ nom, age });
```

## 2.4 Utilisation dans React

Les arrow functions sont **partout** dans React :

```
1 // Composant React
2 const Bouton = ({ texte }) => {
3   return <button>{texte}</button>;
4 };
5
6 // Version courte
7 const Bouton = ({ texte }) => <button>{texte}</button>;
8
9 // Événements
10 <button onClick={() => console.log("Cliqué !")}>
11   Cliquez-moi
12 </button>
13
14 // map() avec arrow function
15 const nombres = [1, 2, 3, 4];
16 const doubles = nombres.map(n => n * 2);
```

## 2.5 Différence avec function

| function               | Arrow Function               |
|------------------------|------------------------------|
| Son propre <b>this</b> | Hérite du <b>this</b> parent |
| Peut être hoistée      | Non hoistée                  |
| Syntaxe plus longue    | Syntaxe plus courte          |

**Pour React** : Utilisez toujours les arrow functions !

## 3 Template Literals (Chaînes Modernes)

### 3.1 Ancienne Méthode : Concaténation

```
1 const nom = "Ahmed";
2 const age = 20;
3
4 // Concaténation avec +
5 const message = "Je m'appelle " + nom + " et j'ai " + age + " ans.";
```

#### Problèmes :

- Difficile à lire
- Risque d'oublier des espaces
- Verbeux

### 3.2 Nouvelle Méthode : Template Literals

```
1 const nom = "Ahmed";
2 const age = 20;
3
4 // Template literal avec backticks ``
5 const message = `Je m'appelle ${nom} et j'ai ${age} ans.`;
6
7 // Expressions à l'intérieur
8 const resultat = `5 + 3 = ${5 + 3}`; // "5 + 3 = 8"
```

#### Syntaxe :

- Utiliser des backticks `` (pas de guillemets)
- Insérer variables/expressions avec \${...}

### 3.3 Chaînes Multi-Lignes

```
1 // Ancien : concat avec \n
2 const ancien = "Ligne 1\nLigne 2\nLigne 3";
3
4 // Nouveau : template literal multi-ligne
5 const moderne = `
6   Ligne 1
7   Ligne 2
8   Ligne 3
9 `;
```

### 3.4 Utilisation dans React

```
1 // Affichage dynamique
2 const Carte = ({ nom, prix }) => (
3   <div>
4     <h3>{nom}</h3>
5     <p>{`Prix: ${prix} DH`}</p>
6   </div>
7 );
8
9 // Classes CSS dynamiques
```

```
10 const Bouton = ({ actif }) => (  
11   <button className={`bouton ${actif ? 'actif' : ''}`}>  
12     Cliquez  
13   </button>  
14 );
```



## 4 Destructuring (Déstructuration)

### 4.1 Destructuring d'Objets

Ancien :

```
1 const personne = { nom: "Ahmed", age: 20, ville: "Agadir" };
2
3 const nom = personne.nom;
4 const age = personne.age;
5 const ville = personne.ville;
```

Moderne :

```
1 const personne = { nom: "Ahmed", age: 20, ville: "Agadir" };
2
3 // Destructuring
4 const { nom, age, ville } = personne;
5
6 console.log(nom); // "Ahmed"
7 console.log(age); // 20
8 console.log(ville); // "Agadir"
```

### 4.2 Renommer lors du Destructuring

```
1 const personne = { nom: "Ahmed", age: 20 };
2
3 // Renommer
4 const { nom: prenom, age: annees } = personne;
5
6 console.log(prenom); // "Ahmed"
7 console.log(annees); // 20
```

### 4.3 Valeurs par Défaut

```
1 const personne = { nom: "Ahmed" };
2
3 // age n'existe pas, on donne une valeur par défaut
4 const { nom, age = 18 } = personne;
5
6 console.log(age); // 18
```

### 4.4 Destructuring de Tableaux

```
1 const fruits = ["Pomme", "Banane", "Orange"];
2
3 // Ancien
4 const premier = fruits[0];
5 const deuxieme = fruits[1];
6
7 // Moderne
8 const [premier, deuxieme, troisieme] = fruits;
9
```

```
10 console.log(premier); // "Pomme"
11 console.log(deuxieme); // "Banane"
```

## 4.5 Utilisation dans React (TRÈS IMPORTANT)

```
1 // Props destructuring (LA technique la plus utilisée)
2 const Carte = ({ nom, prix, image }) => (
3   <div>
4     <img src={image} alt={nom} />
5     <h3>{nom}</h3>
6     <p>{prix} DH</p>
7   </div>
8 );
9
10 // State destructuring
11 const [taches, setTaches] = useState([]);
12 const [count, setCount] = useState(0);
```

## 5 Spread Operator (...)

### 5.1 Avec les Tableaux

```
1 const fruits = ["Pomme", "Banane"];
2 const legumes = ["Carotte", "Tomate"];
3
4 // Ancien : concat
5 const aliments = fruits.concat(legumes);
6
7 // Moderne : spread
8 const aliments = [...fruits, ...legumes];
9
10 // Ajouter un élément
11 const nouveauFruits = [...fruits, "Orange"];
12
13 // Copier un tableau
14 const copie = [...fruits];
```

### 5.2 Avec les Objets

```
1 const personne = { nom: "Ahmed", age: 20 };
2
3 // Copier un objet
4 const copie = { ...personne };
5
6 // Ajouter/Modifier des propriétés
7 const personneComplete = {
8   ...personne,
9   ville: "Agadir",
10  age: 21 // Écrase la valeur précédente
11 };
12
13 console.log(personneComplete);
14 // { nom: "Ahmed", age: 21, ville: "Agadir" }
```

### 5.3 Utilisation dans React (ESSENTIEL)

```
1 // Ajouter un élément à un tableau d'état
2 const [taches, setTaches] = useState([]);
3
4 const ajouterTache = (nouvelleTache) => {
5   setTaches([...taches, nouvelleTache]);
6   // Copie l'ancien tableau + ajoute le nouveau
7 };
8
9 // Modifier un objet dans le state
10 const [user, setUser] = useState({ nom: "Ahmed", age: 20 });
11
12 const changerAge = () => {
13   setUser({ ...user, age: 21 });
14   // Garde nom, change age
15 };
```

**Règle d'Or React** : Ne JAMAIS modifier directement le state, toujours créer une copie avec le spread operator !

## 6 Méthodes de Tableaux (Array Methods)

### 6.1 map() - Transformer un Tableau

```
1 const nombres = [1, 2, 3, 4, 5];
2
3 // Multiplier chaque nombre par 2
4 const doubles = nombres.map(n => n * 2);
5 // [2, 4, 6, 8, 10]
6
7 // Tableau d'objets
8 const personnes = [
9   { nom: "Ahmed", age: 20 },
10  { nom: "Sara", age: 22 }
11 ];
12
13 const noms = personnes.map(p => p.nom);
14 // ["Ahmed", "Sara"]
```

Utilisation dans React (CRUCIAL) :

```
1 const taches = ["Étudier", "Coder", "Réviser"];
2
3 return (
4   <ul>
5     {taches.map((tache, index) => (
6       <li key={index}>{tache}</li>
7     ))}
8   </ul>
9 );
```

### 6.2 filter() - Filtrer un Tableau

```
1 const nombres = [1, 2, 3, 4, 5, 6];
2
3 // Garder seulement les nombres pairs
4 const pairs = nombres.filter(n => n % 2 === 0);
5 // [2, 4, 6]
6
7 // Tableau d'objets
8 const personnes = [
9   { nom: "Ahmed", age: 20 },
10  { nom: "Sara", age: 17 },
11  { nom: "Ali", age: 25 }
12 ];
13
14 const majeurs = personnes.filter(p => p.age >= 18);
15 // [{ nom: "Ahmed", age: 20 }, { nom: "Ali", age: 25 }]
```

Utilisation dans React :

```
1 // Supprimer une tâche
2 const supprimerTache = (id) => {
3   setTaches(taches.filter(tache => tache.id !== id));
4 };
5
```

```

6 // Afficher seulement les tâches non terminées
7 const tachesActives = taches.filter(t => !t.terminee);

```

### 6.3 find() - Trouver un Élément

```

1 const personnes = [
2   { id: 1, nom: "Ahmed" },
3   { id: 2, nom: "Sara" },
4   { id: 3, nom: "Ali" }
5 ];
6
7 // Trouver la personne avec id = 2
8 const personne = personnes.find(p => p.id === 2);
9 // { id: 2, nom: "Sara" }
10
11 // Si non trouvé : undefined
12 const inconnu = personnes.find(p => p.id === 999);
13 // undefined

```

### 6.4 reduce() - Réduire à une Valeur

```

1 const nombres = [1, 2, 3, 4, 5];
2
3 // Somme de tous les nombres
4 const somme = nombres.reduce((acc, n) => acc + n, 0);
5 // 15
6
7 // Compter les occurrences
8 const fruits = ["pomme", "banane", "pomme", "orange", "banane", "pomme"];
9
10 const compteur = fruits.reduce((acc, fruit) => {
11   acc[fruit] = (acc[fruit] || 0) + 1;
12   return acc;
13 }, {});
14 // { pomme: 3, banane: 2, orange: 1 }

```

### 6.5 Chaînage de Méthodes

```

1 const nombres = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
2
3 // Filtrer les pairs, les doubler, faire la somme
4 const resultat = nombres
5   .filter(n => n % 2 === 0) // [2, 4, 6, 8, 10]
6   .map(n => n * 2)           // [4, 8, 12, 16, 20]
7   .reduce((acc, n) => acc + n, 0); // 60

```

## 7 Modules (Import/Export)

### 7.1 Export

Fichier : utils.js

```
1 // Export nommé
2 export const PI = 3.14159;
3
4 export function additionner(a, b) {
5     return a + b;
6 }
7
8 // Export par défaut (un seul par fichier)
9 export default function multiplier(a, b) {
10     return a * b;
11 }
```

### 7.2 Import

Fichier : main.js

```
1 // Import par défaut
2 import multiplier from './utils.js';
3
4 // Import nommé
5 import { PI, additionner } from './utils.js';
6
7 // Import tout
8 import * as utils from './utils.js';
9
10 // Utilisation
11 console.log(multiplier(5, 3)); // 15
12 console.log(additionner(5, 3)); // 8
13 console.log(PI); // 3.14159
```

### 7.3 Utilisation dans React

```
1 // Import React et hooks
2 import { useState, useEffect } from 'react';
3
4 // Import composants
5 import Header from './components/Header';
6 import Footer from './components/Footer';
7
8 // Import CSS
9 import './App.css';
10
11 function App() {
12     return (
13         <div>
14             <Header />
15             <Footer />
16         </div>
17     );
18 }
```

```
18 }  
19  
20 export default App;
```



## 8 Ternaire et Short-Circuit

### 8.1 Opérateur Ternaire

```
1 // Ancien
2 let message;
3 if (age >= 18) {
4     message = "Majeur";
5 } else {
6     message = "Mineur";
7 }
8
9 // Moderne : ternaire
10 const message = age >= 18 ? "Majeur" : "Mineur";
11
12 // Syntaxe : condition ? valeurSiVrai : valeurSiFaux
```

Utilisation dans React :

```
1 const Statut = ({ age }) => (
2     <p>Vous êtes {age >= 18 ? "majeur" : "mineur"}</p>
3 );
4
5 // Affichage conditionnel
6 const Message = ({ erreur }) => (
7     <div>
8         {erreur ? <p className="erreur">{erreur}</p> : null}
9     </div>
10 );
```

### 8.2 Short-Circuit ()

```
1 // Afficher seulement si condition vraie
2 const Message = ({ afficher, texte }) => (
3     <div>
4         {afficher && <p>{texte}</p>}
5     </div>
6 );
7
8 // Équivalent à :
9 {afficher ? <p>{texte}</p> : null}
```

## 9 Récapitulatif : ES6 pour React

### Concepts Essentiels

1. **const/let** : Toujours utiliser, jamais var
2. **Arrow functions** : Syntaxe moderne, utilisée partout
3. **Template literals** : Interpolation avec `${}`
4. **Destructuring** : Props et state
5. **Spread operator** : Copier tableaux/objets
6. **map()** : Afficher des listes
7. **filter()** : Filtrer des données
8. **Import/Export** : Organiser le code
9. **Ternaire** : Affichage conditionnel